

The Low-Token / Next-Token Problem

Why the Weakest Choice Governs the Strongest Weight

Registry: [\[HOWL-LLM-8-2026\]](#)

Series Path: [\[HOWL-LLM-3-2026\]](#) → [\[HOWL-LLM-7-2026\]](#) → [\[HOWL-LLM-8-2026\]](#)

DOI: 10.5281/zenodo.21303618

Date: July 2026

Domain: LLM Mechanics / Generation Theory

Status: Working Methodology

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic’s Claude Opus 4.8.

1. Why this paper exists

Two earlier papers in this series described consequences without isolating their cause. [\[HOWL-LLM-3-2026\]](#) established that LLM generation is ballistic — launched, not steered — and that the illusion of steering comes from adjusting the launch. [\[HOWL-LLM-7-2026\]](#) established that repeated machine modification erodes the structural coherence of mature software, the decoherency tumbler. Both papers are correct, and both stand on a mechanism at the sampler floor that has been used throughout the series but never lifted out and examined on its own.

This paper lifts it out. The mechanism is small enough to state in a paragraph and permanent enough to survive any amount of scaling. It is responsible for a set of failures that appear unrelated to a user — the code that drifts out of your style, the variable the model will not stop renaming, the security fix that quietly reverts, the confident wrong answer in an unfamiliar domain — and are in fact one failure, seen from different angles.

The mechanism is easiest to reach through a single question, which the paper will answer completely: **what will the next token be, after the least-worst last token?**

The reader is assumed to use LLMs and to understand, at a working level, that generation proceeds one token at a time. No further machinery is assumed. Each concept is built before it is used, and the paper’s claim is not a discovery but an explanation — a mechanism walked from the floor upward, with its consequences enumerated so they can be recognized in the wild.

2. The machine's one move

An LLM performs exactly one operation. Given a sequence of tokens — the context — it produces a probability distribution over the possible next token, and one token is selected from that distribution. That token is appended to the context, and the operation repeats. There is no second mechanism. Everything the model appears to do at a higher level — reasoning, planning, refusing, recalling — is this one move, run in sequence.

Because there is only one move, the quality of a generation is entirely the quality of its individual selections, and the quality of a single selection is governed by one underlying quantity: **support**. Support is how much of the training data informs the distribution at this specific point in this specific context. It is the sample size behind the estimate.

Support is not uniform, and its non-uniformity is the entire subject of this paper. Across different contexts, support varies by many orders of magnitude. In some contexts the model is estimating a next-token distribution from effectively the whole corpus; in others, from a handful of examples; in others, from nothing that resembles the current situation at all. The model's behavior is radically different in these regimes, and it gives the user no signal about which regime it is in. The output looks the same either way. That last fact — uniform-looking output across wildly non-uniform support — is where most practical trouble begins, and the paper returns to it repeatedly.

Begin with the regime where everything works.

3. The landslide

The deepest statistics in the entire corpus are the shallowest ones: pair statistics — which token tends to follow which. Every adjacent pair of tokens ever written contributes a sample to these counts, so the evidence behind them is denser than the evidence behind anything else the model knows. This is why, of everything in the context window, the **immediately prior token carries the strongest local conditioning weight**. Attention reads the whole window and integrates far-flung context, but the local pull exerted by the last token is backed by the most samples the model has for anything, and it dominates the shape of the next distribution more than any single other feature.

The clean demonstration is the fragment “**straw-**”. Ask what token comes next and there is, functionally, no distribution to speak of: “berry” takes almost all the probability mass, and “berries,” “man,” “-colored,” and a scattering of others divide the remainder. Millions of samples inform this one selection. It cannot go wrong. The prior token has all but determined the next.

Call this regime the **landslide**: dense support, a sharp peak, the next token nearly forced and nearly always correct. The overwhelming majority of tokens in the overwhelming majority of generations are landslides. This is not a weakness — it is why generation works at all. A machine that produced fluent, grammatical, mostly-correct text could do so only if most of its selections were nearly forced, and they are. The landslide is the model's home ground, and on its home ground it is superb.

The paper is not about the landslide. It is about what happens when the model leaves it.

4. Determinism is not the cure

Before descending into the sparse regime, one exit must be closed, because it is the exit most readers reach for first.

The natural intuition is that the trouble is *randomness*. The model samples from a distribution; sampling introduces chance; therefore reduce the chance. Set temperature to zero and take the most probable token every time. Or go further than temperature — the author’s own work removes the last sources of nondeterminism from the arithmetic itself. Deployed LLM inference is not actually deterministic even at temperature zero: floating-point addition is non-associative on parallel hardware, so reduction order changes the low bits; batching makes one request’s computation depend on which other requests share its batch; mixture-of-experts routing can flip on epsilon differences; serving stacks differ across machines. The VDR series ([[HOWL-VDR-1-2026](#)] and following) replaces floating-point arithmetic with exact integer-rational arithmetic and achieves bit-identical output on every run, in transformers and in diffusion models alike. Determinism, at every level, is achievable. The author has achieved it.

It cures nothing that this paper is about.

Determinism makes a selection *reproducible*. It does not make the underlying distribution *better estimated*. A deterministic model that walks into a badly-mapped region of the corpus takes the same wrong turn on every run — a frozen error instead of a variable one. The exact-arithmetic experiments demonstrate this directly: the streams become perfectly reproducible, and they contain exactly the same class of failure they contained before, now repeatable to the bit. This is worth stating as a finding, because it retires a common hope: **determinism was never the disease**. The disease is not that the model chooses randomly among candidates. The disease is that, in some regions, all the candidates are bad, the model cannot tell, and it must choose anyway. That is the next section.

5. The least-worst token

Some contexts are rare. Your codebase uses a pattern almost no one else uses. A constraint combination arises that the corpus barely contains. A technical corner is reached that few people have ever written about publicly. A convention is invoked that exists, in its exact form, in one repository on Earth — yours. In these regions, the next-token distribution is not a landslide. It is a smear: a small set of candidate tokens, each backed by thin evidence, the estimates poor because the corpus scarcely visited this territory. The peak is low and the shoulders are flat. The model does not know what comes next, in the only sense a model can be said to know anything — the distribution it computed is not concentrated.

And the model must emit something. The architecture has no abstain, no “I am not sure,” no request for clarification issued from inside the forward pass. It computes a distribution and a token comes out. So it selects the best of a bad shortlist — the **least-worst token**: the most probable candidate in a set where even the most probable candidate is not well-supported. Frequently this token is not what the context actually called for. It is not your variable name; it is the corpus’s name for that kind of object. It is not the struct field

your system defines; it is the field a similar-looking system would have. It is the nearest thing the thin evidence could suggest, which in a sparse region is a thing borrowed from a denser region nearby.

A single least-worst token, in isolation, is survivable. A user might not even notice it. The problem is not the token. The problem is everything that comes after it, and to see why requires one more mechanical fact — the one the whole paper turns on.

6. The laundering step

Here is the mechanical heart of the low-token/next-token problem, and it is a single sentence with heavy consequences: **the least-worst token, once emitted, enters the context with full authority.**

Consider what the context window actually is. It is a sequence of tokens. It is *not* a sequence of tokens each annotated with the confidence that produced it. When the model selected the least-worst token, that selection came out of a low, flat, five-candidate smear — a distribution that, if it could speak, would have said “I have very little to go on here.” But the distribution does not travel forward. Only the token does. And once the token is in the context, it is indistinguishable, in every respect that affects the next selection, from a landslide token that came out of a million-sample peak. The context contains “the token that was chosen,” not “the token that was chosen, and here is how unsure the model was when it chose it.”

There is no channel — none, at any model scale, at any temperature, in any current architecture — by which the uncertainty that produced a token propagates to the tokens that follow it. The forward pass has no memory of its own doubt. The posterior uncertainty is *laundered into prefix fact* at the instant of emission. The model has its doubt, emits, and one token later has forgotten it entirely, because there was never anywhere for that doubt to be stored.

Now combine this with Section 3. The strongest local conditioning weight on the next selection is the immediately prior token. So the next selection conditions *hardest* on precisely the token that was least trustworthy — and it conditions on it as though it were certain, because the machine cannot represent that it was anything else.

This is the pivot of the entire mechanism. A model that could carry its uncertainty forward could recover: it could treat a low-confidence token as tentative, hedge the next step, hold the door open for a correction. It cannot. The uncertainty is gone, the token is fact, and the fact is the heaviest single influence on what comes next.

7. Sparse conditions on sparse

Follow the sequence forward from the laundered token.

The context was already in a sparse region — that is what produced the least-worst token in the first place. The token that was emitted is slightly wrong for that region: a median borrowing, an approximate fit. Now ask about the support for the *pair*: the wrong-ish token, followed by whatever should come after it, in this already-rare context. That pair

is rarer than the region was. A rare context that now contains a mildly off token is a *more* specific, *less* sampled situation than the rare context alone. The candidate set for the next token narrows. The estimates degrade. The next selection is a least-worst drawn from a shortlist worse than the one before it.

This is the low-token/next-token problem, stated exactly. **Tokens that follow low-choice tokens are worse selections, because they sequentially inherit a prefix built from low-choice selections — and the strongest weight of all is the prior token.** The error does not merely persist into the next step. It compounds, because each step conditions most heavily on the previous step’s weakest output, and each step’s own uncertainty is laundered before the following step runs. There is no point at which the accumulated doubt becomes visible to the machine and triggers a correction, because the doubt was never accumulated anywhere. Each step begins fresh, treating a compromised prefix as ground truth.

The intuitive form of this, which every experienced user has felt without a mechanism for it: *if the last turn was good enough, why is this turn different?* Because nothing held the last turn in place. There is no invariant, no confidence carried forward — only the next selection, conditioned on a prefix that silently degraded. The generation is not being steered off course by an external force. It is walking off course under its own conditioning, one laundered token at a time.

8. Drainage

The compounding has a shape, and the shape is the reason the failure is so difficult to catch in the act.

Picture the corpus as terrain. The dense-support regions — common idioms, tutorial patterns, the median of everything ever written publicly — are valleys: broad, low, heavily sampled, easy to fall into and hard to climb out of. The sparse regions — your specific system, your unusual requirement, your one-of-one convention — are ridgelines: narrow, high, lightly sampled, easy to fall off. A least-worst token is a step off the ridge. And every subsequent conditional, pulled by the mass of the corpus, flows *downhill* toward the valleys.

Crucially, the stream does not fragment as it falls. The model’s fluency machinery fights fragmentation at every single step: whatever the prefix now contains, however compromised, the model finds a continuation that renders it locally coherent. It is very good at this — local coherence is a landslide-grade skill, backed by enormous support. So the generation does not break. It **bends**. It exits the intended region along a smooth, plausible curve and re-enters the nearest dense basin, where support is thick again and every selection is a landslide once more. The generation *recovers its confidence without recovering its correctness*. It reads well. It reads better, often, than the correct output would have, because the valley is where the most fluent, most idiomatic, most tutorial-shaped text lives.

This is the signature that every practitioner has seen and few have named. A function that begins in your idiom — your naming, your structure, your conventions — reaches the

part of your pattern the corpus does not know, takes one soft wrong turn there, and then proceeds with total fluency in a dialect that is no longer yours. The guard you would never write. The allocation in the wrong style. The renamed variable. The helpful refactor of the thing that must not be refactored. The error is not the wrong turn. The error is that everything after the wrong turn conditioned on it with full weight, and the stream’s own coherence-seeking actively steered toward the valley where that wrong token is normal and your intent is foreign.

Name the phenomenon: **drainage**. Not a random walk — a random walk has no preferred direction, and this has a strong one. The descent is directed, always the same direction: away from the specific intent that lives on the ridge, toward the corpus mass that fills the valley. And under deterministic sampling, it is the *same* descent every run — the exact-arithmetic experiments show a generation draining down the identical path each time, because the terrain is fixed and the walk is now deterministic. Reproducible descent toward the mean.

9. The atomic case: the renaming problem

The smallest complete instance of the mechanism has been observed for years, resists every attempt at instruction, and contains every element of the machinery in miniature.

Name a variable `viewScreen` in a context where the corpus’s overwhelming convention for that kind of object is `viewRect`. Ask the model to work with your code, and watch it rename the variable — not once, correctable, but persistently, reverting your name every time it must produce the identifier, regardless of how forcefully you forbid it.

The mechanism is exactly the one this paper has built. The identifier is not *stored* anywhere as a fact the model holds. Every time `viewScreen` must appear in the output, it is *reconstructed* — the model reaches the position where the name goes and runs the same next-token election it runs everywhere. At that position, two forces vote. In-context evidence: your code, right there in the window, saying `viewScreen`, a distribution informed by a single sample. Prior mass: a million repositories where an object of that shape is named `viewRect`, a landslide. Copy-from-context is a real and strong behavior — the model does mostly repeat identifiers it sees — but it is a *weighting*, not a rule. It is one force among two, and where your name is semantically “wrong” by corpus standards, the two forces are close enough that the election is genuinely contested. It only has to lose once. And it is held dozens of times per file, once at every appearance of the name.

The moment it loses once, the laundering step (Section 6) takes over and the loss cascades. The median token `viewRect` enters the context with full authority. Now the context contains *both* names, and the median one has in-context support too. Every subsequent election is conditioned on a contaminated electorate, and the model is not “changing your name back” — it is re-deriving the name at every use site, and the derivation now has evidence on both sides, pulling harder toward the corpus with each appearance it wins. Drainage, at the granularity of a single identifier.

This is why enforcement language fails, and it fails *structurally*, not for lack of emphasis.

“NEVER rename these variables — many modules depend on them, renaming breaks the build” is, mechanically, more tokens in the context. Those tokens shift the weighting at each election. They do not remove the election. Removing the election would require a rejection step — a mechanism that checks the proposed token against a constraint and refuses violations — and the forward pass has none (Section 6). Putting the rule in the prompt is an attempt to install an invariant through the context window, and the context window is an *evidence channel for an estimator*, not an invariant store. You can add evidence. You cannot add a constraint. Berating the model is adjusting a prior with a rock.

The available outcomes are exactly two, and there will never be a third within this architecture. Either rename your variable to `viewRect` — dissolve the conflict by moving your code onto the corpus’s manifold, so that prior and context vote together and the election becomes unanimous — which works, at the cost of a small payment of *yours-ness* each time, sanding off the specificity that made your code navigable and yours. Or take the artifact out of the sampler’s jurisdiction entirely: generate once, own it, and never submit it to another election. **Rename it, or own it.** The whole of this paper is contained in that variable.

10. Where sparse support lives

The mechanism would be a curiosity if sparse regions were rare, or unimportant, or shrinking. They are none of these. They are where all the value is, and they are inexhaustible.

The intuitive hope is that scaling solves this: a larger model trained on a larger corpus has denser support everywhere, so the sparse regions fill in and the least-worst token becomes rare. Scaling does densify the manifold — it genuinely improves coverage of the common. But it cannot reach the regions that matter most, for a structural reason: **the valuable regions are valuable precisely because they are rare.** Your codebase exists once. Your architectural conventions exist once. A genuine fix for a real security vulnerability is off-median *by construction* — the vulnerability existed *because* the natural, corpus-dense way to write that check was subtly wrong, so the fix is necessarily a departure from the dense pattern (see [HOWL-COMP-14-2026] on regression to the vulnerable mean). A genuinely new system, an unprecedented constraint, a first-of-its-kind design: each one mints new off-manifold territory that no prior corpus could contain. **Specificity generates the sparse tail faster than any training run absorbs it.** The tail is not a shrinking remainder that scaling will eventually eliminate. It is continuously replenished by exactly the acts — building the new, fixing the specific, holding a convention — that constitute original technical work. The machine is strongest where you need it least, and weakest at the precise coordinates of everything that makes your work yours.

The full escalation, from safe to impossible:

Dense support — millions of samples. Landslides; stable, correct streams. Common idioms, tutorial patterns, mainstream framework code, genre artifacts. The model’s home ground.

Thin support — hundreds of samples. Least-worst picks that enter the context with full

authority. Uncommon APIs, niche conventions, the first sign of trouble.

Sparse-on-sparse — pair-level support after a least-worst token. Compounding; the drainage begins. This is the interior of a generation working through your specific code.

Off-manifold — one-of-one. No meaningful estimate exists; the model snaps to the nearest dense region and drains toward it. Your codebase, your fix, your convention — and, outside software, your face against a matcher trained on a distribution you have left ([[HOWL-COMP-14-2026](#)] treats the identity-drift case).

Zero support — self-invented. A dialect whose only sample is its own just-emitted definition. Ask the model to invent a macro language and then write programs in it: the language has, at the moment of use, a corpus of exactly one document — the definition it produced four hundred tokens ago. Its only mechanism for using the language correctly is in-context imitation, which is soft, decays with distance, and bends steadily toward the raw-corpus mass where the macro does not exist. Bootstrap incoherence: the language, its implementation, and its entire usage corpus must all be generated in one mutually-conditioning stream, with no external verifier anywhere in it.

One inversion completes the terrain map, because it shows that even dense support does not contain what would actually rescue the sparse case. Dense regions have their own blindness. Where support is overwhelming, sequences fuse into single statistical units and their *internal structure* becomes invisible. The well-known failure to count the letters in “strawberry” is not a sparse-support failure — it is the opposite. “berry” after “straw-” is so forced, so landslide, that the word passes through the model as one fused object, and its component letters were never load-bearing in any conditional the model ran. Thin support cannot find your structure; dense support cannot see structure at all, only flow. And neither regime contains *holding* — the carrying of a fact across positions, checked and enforced. This is the finding of the companion papers restated at the token floor: the architecture has no rejection step, and — this paper’s specific contribution — **no confidence channel**. The machine can neither refuse a proposed token nor remember that it was unsure of one. Both absences are permanent for the architecture, and between them they are the entire low-token/next-token problem.

11. Containments

The mechanism cannot be fixed. It is architectural, and this paper’s final claim is that no amount of scaling touches it. But it can be *routed around*, and four containments are documented across this registry, in ascending order of strength. Each one accepts that the machine forgets its doubt, and works around the forgetting rather than curing it.

Manufacture density. If the sparse region is where failure lives, make the region locally dense for the duration of one generation. Pack the context with owned material: the struct definitions, the named existing patterns the new code should follow, the specification written from ownership. This performs, by hand, a kind of kernel-density estimation around your own conventions — for one generation, your off-manifold territory has enough in-context evidence to shift the elections in your favor, and they run with the best odds they will ever have. This is why one carefully-assembled prompt outperforms a

hundred corrective exchanges: the corrections arrive after the drainage, while the manufactured density arrives before it. (Specification: [HOWL-ENG-3-2026] Appendix F.1; [HOWL-COMP-14-2026] context assembly.)

One pass. Manufactured density is a wasting asset, and this is the containment users most often violate. Every token the model *generates* and appends to the context is median material that dilutes your owned evidence; every subsequent turn re-runs every election with a progressively more contaminated electorate. Multi-turn refinement of owned code is, mechanically, a daily referendum on every identifier and every convention in the file, held against an electorate of everything ever written, with each turn’s median output added to the voter rolls. The renaming problem is unwinnable across turns for exactly this reason. Generate once, at maximum attention, against maximum manufactured density — and then stop submitting the artifact to the sampler.

External certainty. Since the machine has no memory of its own confidence, supply one from outside. Every load-bearing property must live in machinery that actually holds things: deterministic verifiers, pinned tests, type systems, checked runtime invariants, deterministic code generators. The generator proposes; the machinery rejects violations; the rejection step the architecture lacks is bolted on externally. This is the whole strategy of the composition method in [HOWL-COMP-14-2026] and the conversion criterion in [HOWL-ENG-3-2026] Appendix B: never ask the model to hold an invariant, because it cannot; put every invariant in a machine that can.

Ownership on arrival. The strongest and most permanent containment is the artifact’s exit from the sampler’s jurisdiction. Integrate the output, adjust it, debug it, commit it in your own words — build position in it ([HOWL-ENG-3-2026] Steps 3 and 6) — and it enters stillness, where no election is ever held over it again. Drainage operates only on what the sampler can still touch; code that has become yours is no longer in play. This is why the durable protocol is generate-once-then-own: the generation is the model’s, the ownership is yours, and the boundary between them is patrolled by the act of integration.

The honest boundary must be stated plainly, because it is the paper’s final claim. All four containments are *routing*, not repair. The mechanism is in the architecture. Scale the model a hundred-million-fold and every estimate improves, the sparse regions recede somewhat, the least-worst tokens grow rarer — and nothing structural changes. The forward pass will still lack a rejection step. The context will still be a sequence of tokens rather than of confidences. The prior token will still carry the strongest weight. And a least-worst token will still be laundered into fact one step after it was doubted, in whatever sparse regions remain — and Section 10 established that sparse regions are replenished by original work faster than scaling drains them. The problem is permanent for this architecture. A different architecture — one with an actual rejection step and an actual confidence channel — would be a different machine, and building it is a different program (see [HOWL-LLM-1-2026] and the VDR-LLM-Prolog series, where the rejection step is provided by construction through a symbolic knowledge base). That program is the only thing that changes the answer to this paper’s question.

12. Closing

The question was: what will the next token be, after the least-worst last token?

The complete mechanical answer: **the least-worst continuation of a prefix that now contains an error wearing the costume of a fact — chosen from a smaller candidate set, with thinner support than the token before it, pulled downhill by the mass of the corpus, and delivered with exactly the same fluency as everything else in the stream.** The error is invisible in the output because the output is uniformly confident; the descent is directed because the corpus has a center of mass; and the whole process is deterministic under exact arithmetic, which proves that the trouble was never randomness but always the machine’s inability to know, or to remember, that it did not know.

Everything else built across this registry’s LLM and engineering series is machinery against that one sentence. The tumbler’s deterministic verifiers, the pinned invariants, the one-pass discipline, the ownership protocol, the human held at the end of every accountability chain, the symbolic rejection step of the VDR-LLM-Prolog architecture — all of it is a single intervention applied at different altitudes and different scales. It is, in every case, the same thing: **an external memory of certainty, built around a machine that forgets its own doubt one token after having it.**

Appendix A — The Support Escalation Ladder

Regime	Support depth	Selection behavior	Stream effect	Example
Landslide	Millions of samples	Near-forced, near-always right	Stable, correct	“straw-” → “berry”; tutorial idioms; mainstream framework code
Thin	Hundreds of samples	Least-worst pick; enters with full authority	First soft wrong turn	Uncommon API; niche convention
Sparse-on-sparse	Pair-level, post-least-worst	Worse least-worst from a narrower set	Compounding; drainage begins	Interior of a generation working through your specific code

Regime	Support depth	Selection behavior	Stream effect	Example
Off-manifold	One-of-one	Snap to nearest dense region	Full drainage; dialect shift	Your codebase; a real security fix; the renaming problem; identity drift
Zero-support	Only its own definition	In-context imitation, decaying with distance	Bootstrap incoherence	A self-invented macro language, written in itself
Dense-fused (inverse)	Overwhelming	Sequence passes as one fused unit	Internal structure invisible	Counting the letters in “strawberry”

Appendix B — The Drainage Trace

A single generation over owned code, step by step.

Step	Context state	Support regime	Selection	Effect on stream
1	Owner’s idiom; context packed with owned structs and named patterns	Manufactured density	Landslides	Correct; in your dialect
2	Reaches the one-of-one pattern — the unusual guard, the owned name	Off-manifold	Least-worst: the nearest median equivalent	Soft wrong turn; enters context with full authority
3	Prefix now contains the median token	Sparse-on-sparse	Conditions hardest on the least-trustworthy token	Error compounds; no doubt carried forward
4	Stream bends toward the dense basin	Rising toward dense	Landslides — in the <i>wrong</i> dialect	Fluency fully recovered; correctness not

Step	Context state	Support regime	Selection	Effect on stream
5	Remainder of generation	Dense	Confident, idiomatic median continuation	The building looks perfect; it is no longer yours

Appendix C — Containment Cross-Reference

Containment	This paper	Full specification elsewhere in the registry
Manufacture density	§11.1	[HOWL-ENG-3-2026] App. F.1 (phase 3); [HOWL-COMP-14-2026] context assembly
One pass	§11.2	[HOWL-ENG-3-2026] App. F.3 (multi-turn erosion); [HOWL-LLM-2-2026] [HOWL-COMP-14-2026] verifier stack;
External certainty	§11.3	[HOWL-ENG-3-2026] App. B (conversion criterion)
Ownership on arrival	§11.4	[HOWL-ENG-3-2026] Steps 3, 6 (position, the log); [HOWL-COMP-14-2026] two-class theorem
Architectural replacement (out of scope here)	§11 boundary	[HOWL-LLM-1-2026]; VDR-LLM-Prolog series (rejection step by construction)

Appendix D — Relation to Prior LLM-Series Papers

Paper	Establishes	This paper adds
[HOWL-LLM-3-2026] Riding the Rocket	Generation is ballistic, not steerable	The token-level mechanism <i>of</i> ballistic drift: laundering + prior-token dominance

Paper	Establishes	This paper adds
[HOWL-LLM-4-2026] Incompatibility by Construction	What cannot be fixed with more LLM	The specific unfixable: no confidence channel, permanent for the architecture
[HOWL-LLM-7-2026] Agentic Decoherency Tumbler	Repeated modification erodes coherence	The per-token cause of per-edit erosion: every regeneration re-runs the elections
[HOWL-LLM-1-2026] Integer LLM + Prolog	A neural-symbolic architecture	Names precisely what the symbolic side supplies: the missing rejection step

Appendix E — Complete Point Enumeration

Every distinct claim developed on this topic, whether or not it reached the prose. Marked: ✓ in paper, ✂ cut for flow, ✎ new to appendix.

E.1 The One Move and Support (Claims 1–12)

#	Claim	Status
1	An LLM has exactly one operation: distribution over next token, select, append, repeat	✓ §2
2	Everything higher-level (reasoning, refusing, recalling) is this one move in sequence	✓ §2
3	Generation quality is entirely the quality of individual selections	✓ §2
4	Selection quality is governed by <i>support</i> — sample size behind the estimate	✓ §2
5	Support is non-uniform across contexts by many orders of magnitude	✓ §2
6	The model gives the user no signal about which support regime it is in	✓ §2

#	Claim	Status
7	Output looks uniform across wildly non-uniform support — where trouble begins	✓ §2
8	Pair (bigram) statistics are the densest in the corpus — every adjacent pair contributes a sample	✓ §3
9	Therefore the immediately prior token carries the strongest local conditioning weight	✓ §3
10	Attention reads the whole window, but the prior token’s local pull has the most samples behind it	✓ §3
11	The landslide: “straw-” → “berry”, near-forced, near-always right	✓ §3
12	Most tokens in most generations are landslides — this is why generation works at all	✓ §3
12a	Support depth and <i>distance</i> interact: the prior token dominates locally, but long-range context can still redirect — the landslide is local, not global	*
12b	“Home ground” framing: on dense support the model is superb; the paper is about what happens when it leaves home ground	✓ §3

E.2 Determinism (Claims 13–20)

#	Claim	Status
13	The naive fix is “reduce randomness” — set temperature to zero	✓ §4

#	Claim	Status
14	Deployed temp-0 isn't deterministic: FP non-associativity, batching, MoE routing, serving-stack variance	✓ §4
15	VDR exact integer-rational arithmetic achieves bit-identical output, transformers and diffusion	✓ §4
16	Determinism at every level is achievable — the author has achieved it	✓ §4
17	Determinism makes selection <i>reproducible</i> , not the distribution <i>better estimated</i>	✓ §4
18	A deterministic model takes the <i>same</i> wrong turn every run — a frozen error	✓ §4
19	The exact-arithmetic streams contain the same class of failure, now bit-reproducible	✓ §4
20	Determinism was never the disease	✓ §4
20a	Corollary: reproducibility is a <i>debugging</i> asset even though it fixes nothing — the same drainage path can be studied	*
20b	The temperature knob trades one failure for another: high temp adds exploration noise, low temp freezes the drainage; neither adds a confidence channel	*

E.3 The Least-Worst Token (Claims 21–28)

#	Claim	Status
21	Some contexts are rare: your pattern, an unusual constraint combination, a niche corner, a one-repo convention	✓ §5
22	In rare regions the distribution is a smear: small candidate set, thin evidence, low flat peak	✓ §5
23	“Does not know” in the only sense a model can — the computed distribution is not concentrated	✓ §5
24	The architecture has no abstain, no “I am not sure,” no clarification from inside the forward pass	✓ §5
25	It selects the best of a bad shortlist — the least-worst token	✓ §5
26	Frequently the least-worst token is a <i>borrowing from a denser region nearby</i>	✓ §5
27	A single least-worst token in isolation is survivable; a user may not notice	✓ §5
28	The problem is not the token; it is everything after it	✓ §5
28a	The least-worst token is <i>systematically biased toward the dense neighbor</i> , not randomly wrong — this directionality is what makes drainage directed rather than diffuse	*
28b	Sparse-region emission is where “hallucination” is manufactured: a confident token with no support is the atom of a confident falsehood	*

E.4 The Laundering Step (Claims 29–36)

#	Claim	Status
29	The least-worst token enters the context with <i>full authority</i> — the mechanical heart	✓ §6
30	The context is a sequence of tokens, NOT a sequence of (token, confidence) pairs	✓ §6
31	No channel exists — any scale, any temperature, any current architecture — for uncertainty to propagate forward	✓ §6
32	Posterior uncertainty is laundered into prefix fact at the instant of emission	✓ §6
33	The model forgets its own doubt one token after having it — there was nowhere to store it	✓ §6
34	The next selection conditions <i>hardest</i> on the least-trustworthy token (prior-token dominance)	✓ §6
35	And conditions on it as certain, because the machine cannot represent it as anything else	✓ §6
36	A model that could carry uncertainty forward could recover (hedge, hold the door open) — it cannot	✓ §6
36a	Chain-of-thought does NOT solve this: reasoning tokens are also laundered — a low-confidence reasoning step becomes a certain premise for the next step	*

#	Claim	Status
36b	Sampling <i>many</i> times (self-consistency) partially routes around it by averaging over which least-worst token got laundered — but each individual chain still drains	*
36c	The absence is symmetric to the rejection-step absence: no rejection = cannot refuse a proposal; no confidence channel = cannot doubt one it accepted	*

E.5 Sparse-on-Sparse Compounding (Claims 37–43)

#	Claim	Status
37	The context was already sparse — that produced the least-worst token	✓ §7
38	The pair (wrong-ish token → continuations, in a rare context) is rarer than the region alone	✓ §7
39	Candidate set narrows, estimates degrade, next least-worst is worse	✓ §7
40	Tokens following low-choice tokens are worse — they inherit a prefix of low-choice selections	✓ §7
41	Error compounds because each step conditions most on the previous step’s weakest output	✓ §7
42	Each step’s uncertainty is laundered before the next runs — no accumulation of visible doubt	✓ §7

#	Claim	Status
43	Intuitive form: “if the last turn was good enough, why is this turn different?” — nothing held it	✓ §7
43a	The compounding is super-linear in sparse regions: each laundered token <i>deepens</i> the rarity, so degradation accelerates rather than staying constant	*
43b	There is a recovery case: if long-range context is strong enough, the stream can re-anchor to it — but only where the owned material is dense enough in-context to outvote the drift (this is the mechanism manufactured-density exploits)	*

E.6 Drainage (Claims 44–52)

#	Claim	Status
44	The corpus as terrain: dense regions are valleys, sparse regions are ridgelines	✓ §8
45	A least-worst token is a step off the ridge; every subsequent conditional flows downhill	✓ §8
46	The stream doesn’t fragment — fluency machinery renders any prefix locally coherent	✓ §8
47	So it <i>bends</i> : exits the intended region on a smooth curve, re-enters the nearest dense basin	✓ §8
48	Recovers confidence without recovering correctness	✓ §8

#	Claim	Status
49	Reads <i>better</i> than the correct output would — the valley holds the most idiomatic text	✓ §8
50	The signature: starts in your idiom, one soft wrong turn, proceeds fluently in a dialect no longer yours	✓ §8
51	Named: drainage — directed (toward corpus mass), not random walk	✓ §8
52	Under exact arithmetic: the <i>same</i> drainage path every run — reproducible descent to the mean	✓ §8
52a	Drainage explains why longer generations are more dangerous than shorter ones: more steps = more opportunities to step off a ridge, and no step ever climbs back deliberately	*
52b	Drainage explains the “regression to tutorial” effect: unusual architectures decay toward the most-tutorialized version of themselves	*
52c	The valley is not neutral — it is the <i>median of all public code/text</i> , which encodes the median’s biases, the median’s security posture, the median’s staleness	*

E.7 The Renaming Problem (Claims 53–62)

#	Claim	Status
53	The identifier is reconstructed at every position, not stored as a fact	✓ §9
54	Each appearance is a fresh election: one-sample context vs. million-repo prior	✓ §9
55	Copy-from-context (induction) is strong but <i>statistical</i> — a weighting, not a rule	✓ §9
56	Where the name is “wrong” by corpus standards, the election is genuinely contested	✓ §9
57	Held dozens of times per file; it only has to lose once	✓ §9
58	After one loss the flip cascades — the median name gains in-context support too	✓ §9
59	The model isn’t “changing it back” — it re-derives at every site with two-sided evidence	✓ §9
60	Enforcement language fails structurally: rules are tokens, they shift weighting, never remove the election	✓ §9
61	Installing an invariant through the context window mistakes an evidence channel for an invariant store	✓ §9
62	Two outcomes only: rename to the median (pay yours-ness) or own the artifact. Rename it, or own it	✓ §9

#	Claim	Status
62a	The same mechanism governs <i>style</i> drift, not just names: brace placement, comment density, error-handling idiom — each is an election against the corpus median	*
62b	It also governs <i>API surface</i> drift: the model reaches for the corpus-standard method name over your custom wrapper’s name	*
62c	Larger context windows make this <i>worse</i> , not better, past a point: more generated median tokens accumulate in-window to outvote the original owned sample	*

E.8 Where Sparse Support Lives (Claims 63–74)

#	Claim	Status
63	Off-manifold is where all value is	✓ §10
64	Scaling densifies the manifold but cannot reach what’s valuable-because-rare	✓ §10
65	Your codebase exists once; your conventions exist once	✓ §10
66	A real security fix is off-median by construction (the bug was the dense idiom)	✓ §10
67	New systems, unprecedented constraints, first-of-kind designs mint fresh off-manifold territory	✓ §10
68	Specificity generates the sparse tail faster than any training run absorbs it	✓ §10

#	Claim	Status
69	The tail is continuously replenished by exactly the acts that constitute original work	✓ §10
70	The machine is strongest where you need it least, weakest at the coordinates of everything yours	✓ §10
71	The ladder: dense → thin → sparse-on-sparse → off-manifold → zero-support	✓ §10
72	Zero-support: a self-invented dialect with a corpus of one document (its own definition)	✓ §10
73	Dense-region inverse blindness: fused units, internal structure invisible (the “strawberry” letters)	✓ §10
74	Neither regime contains <i>holding</i> : no rejection step, no confidence channel — both permanent	✓ §10
74a	There is a <i>middle</i> danger zone rarely discussed: “plausible-dense” — regions dense enough to be confident, wrong enough to matter (e.g., a deprecated-but-heavily-tutorialized API)	*
74b	Retrieval augmentation is a density-manufacture strategy at the corpus level — it works by the same mechanism as §11.1, and fails the same way when the retrieved material is itself median	*

#	Claim	Status
74c	Fine-tuning on your own code is manufactured density baked into weights — durable, but it moves your convention onto the model’s manifold, not the reverse; it is a scaled version of “rename to the median”	*

E.9 Containments (Claims 75–84)

#	Claim	Status
75	The mechanism cannot be fixed, only routed around	✓ §11
76	Manufacture density: pack owned context = manual kernel-density estimation for one generation	✓ §11.1
77	One carefully-assembled prompt beats a hundred corrections — corrections arrive after drainage	✓ §11.1
78	One pass: manufactured density is a wasting asset; generated median tokens dilute it	✓ §11.2
79	Multi-turn = a daily referendum on every identifier against everything ever written	✓ §11.2
80	External certainty: put every invariant in machinery that holds things (the bolted-on rejection step)	✓ §11.3
81	Ownership on arrival: integrate until positioned; the artifact exits the sampler’s jurisdiction	✓ §11.4
82	All four are routing, not repair	✓ §11

#	Claim	Status
83	Scaling improves estimates, recedes sparse regions, changes nothing structural	✓ §11
84	A rejection step + confidence channel = a different machine (VDR-LLM-Prolog) — the only thing that changes the answer	✓ §11
84a	The four containments compose multiplicatively: density $\times$ one-pass $\times$ external-verify $\times$ ownership; skipping any one reopens the drainage at that layer	*
84b	Containments have a cost: each pays in human attention (assembly, verification, integration) — the mechanism is not free to route around, which is why “just use the model” underprices real work	*

E.10 The Answer (Claims 85–88)

#	Claim	Status
85	The answer: the least-worst continuation of a prefix containing an error costumed as fact — smaller set, thinner support, pulled downhill, delivered with equal fluency	✓ §12
86	The error is invisible because the output is uniformly confident	✓ §12
87	The descent is directed because the corpus has a center of mass	✓ §12

#	Claim	Status
88	Everything in the registry is one intervention: an external memory of certainty around a machine that forgets its doubt one token after having it	✓ §12

Appendix F — The Two Architectural Absences

The paper’s deepest structural claim, tabulated: two missing mechanisms, their consequences, and what would supply each.

Property	No Rejection Step	No Confidence Channel
What’s missing	A mechanism to check a proposed token against a constraint and refuse violations	A mechanism to carry a selection’s uncertainty forward to condition later selections
Where it would live	Between distribution and emission	In the context representation (token → token+confidence)
Failure it causes	Cannot hold an invariant; the renaming problem; smuggled test values; violated rules	Cannot recover from a bad token; laundering; drainage; compounding
User-visible symptom	“It keeps doing the thing I told it not to”	“It was confidently wrong” / “it drifted”
Prompt-level workaround	None (rules are evidence, not constraints)	None (uncertainty has nowhere to go)
Real containment	External verifier (bolt on the rejection step)	Manufactured density + one-pass (prevent the bad token; don’t submit it again)
Architectural fix	Symbolic constraint layer ([HOWL-LLM-1-2026])	A confidence-carrying representation (open; not in any shipping architecture)
Permanent for transformers?	Yes	Yes

Appendix G — What Common “Fixes” Actually Do

Each popular remedy mapped to the mechanism: what it touches, what it cannot.

Proposed fix	What it actually does	Touches the mechanism?	Residual failure
Lower temperature	Freezes selection to the argmax	No	Frozen drainage; same wrong turn every run
Higher temperature	Adds exploration noise to selection	No	More diverse wrong turns; no confidence added
Bigger model	Densifies the manifold; recedes sparse regions	No	Off-manifold work still drains; tail replenished by new work
Longer context	More room for owned material — <i>and</i> more room for generated median to accumulate	Partially (both directions)	Past a point, worsens the renaming problem
Stronger system prompt	Adds evidence tokens that shift weighting	No	Rules are not constraints; election remains
Chain-of-thought	More tokens, more intermediate structure	No	Reasoning tokens are also laundered
Self-consistency (N samples, vote)	Averages over <i>which</i> least-worst token got laundered	Partially	Each chain still drains; cost scales with N
RAG / retrieval	Manufactures density at corpus level	Routes around (like §11.1)	Fails when retrieved material is itself median
Fine-tuning on your code	Bakes manufactured density into weights	Routes around, durably	Moves your convention onto the manifold, not the reverse
External verifier	Bolts on the missing rejection step	Routes around, at the property level	Cost in verifier construction; can’t verify what’s unpinned
Symbolic hybrid (VDR-LLM-Prolog)	Supplies rejection step by construction	Yes — changes the machine	Different architecture; different program

Appendix H — Manifestations Across Domains

The single mechanism, recognized in surface forms that look unrelated.

Surface manifestation	Domain	Off-manifold trigger	Drainage direction
Renamed variable	Code	Your one-repo convention	Toward corpus-standard names
Style drift	Code	Your idiom (braces, comments, error handling)	Toward the median style
Reverted security fix	Code	The fix is off-median by construction	Back toward the vulnerable idiom
Deprecated-API suggestion	Code	Heavily-tutorialized old API	Toward the most-written, not the current
Confident hallucination	Text/QA	A fact sparsely represented	Toward a plausible dense-neighbor fact
“Regression to tutorial”	Architecture	Your unusual system design	Toward the most-tutorialized version of it
Invented-dialect incoherence	Code	Self-defined macro language (zero-support)	Toward raw base-language corpus
Identity mismatch	Biometrics	A face off the training distribution	Toward “no match” / nearest enrolled pattern
Median cultural default	Any generation	An unusual specified constraint	Toward the statistical center of the corpus

The unifying claim: these are not nine problems. They are one mechanism — a least-worst token, laundered, draining toward corpus mass — observed in nine places.

Appendix I — The Drainage Geometry (Extended Trace)

A finer-grained walk than the paper’s Appendix B, annotating the confidence the model has versus the confidence the context records.

Token position	Region	Model's actual confidence	Recorded in context as	Gap	Consequence
n	Owned pattern, packed	High (manufactured)	Certain	None	Correct token
n+1	Edge of owned pattern	Falling	Certain	Opening	Correct but fragile
n+2	Off-manifold — the unusual guard	Low (smear)	Certain	Maximal	Least-worst emitted; laundering occurs here
n+3	Sparse-on-sparse	Lower	Certain	Compounding	Conditions hard on n+2's error
n+4	Bending toward basin (wrong basin)	Rising (wrong basin)	Certain	Closing — wrongly	Fluent; incorrect
n+5...	Dense basin (wrong dialect)	High	Certain	None	Confident median continuation

The “Gap” column is the paper’s whole subject: the divergence between what the model actually knew and what the context recorded it as knowing. The gap opens at the off-manifold token, is maximal at the moment of laundering, and closes — falsely — as the stream reaches the wrong valley. The model never sees the gap, because the gap lives in exactly the confidence channel the architecture lacks.

Appendix J — Diagnostic Signatures

How to recognize each stage of the mechanism in real output — a field guide.

Signature in output	Stage	What it indicates	Response
Correct, in-idiom, unremarkable	Landslide	Dense support; safe	Proceed
First unfamiliar-but-plausible token	Thin support	Approaching a ridge	Increase manufactured density if this region matters
A convention or name subtly not-yours	Least-worst emitted	Laundering just occurred	Stop; do not continue the turn

Signature in output	Stage	What it indicates	Response
Fluent continuation in a slightly-off dialect	Drainage underway	The stream has bent	Discard from the wrong turn; regenerate with more owned context
Confident, idiomatic, wrong-for-your-system	In the wrong valley	Full drainage complete	Do not correct in-turn (corrections arrive post-drainage); restart one-pass
The model “won’t stop” doing X despite instruction	Repeated election loss	Rule-as-evidence failing	Rename to median OR remove from sampler (own it)
Confident wrong fact	Sparse emission in text	Hallucination atom	Verify externally; the model cannot flag it

Appendix K — The Cost Ledger of Containment

Routing around the mechanism is not free. Each containment’s cost, in the currency it actually charges.

Containment	Cost currency	Who pays	Why it can’t be automated away
Manufacture density	Assembly effort — gathering owned context, worked examples	The human with position	Knowing <i>what</i> is load-bearing to include is a judgment (which patterns, which names)
One pass	Foregone convenience — no iterative chat refinement	The human	Requires discipline against the natural multi-turn workflow; the erosion is invisible per-turn

Containment	Cost currency	Who pays	Why it can't be automated away
External certainty	Verifier construction — tests, types, invariants, generators	The human, upfront	The verifier encodes what “correct” means; the model can't supply this (it's the missing rejection step)
Ownership on arrival	Integration labor — reading, adjusting, debugging until positioned	The human	Position is built only by contact ([HOWL-ENG-3-2026]); it cannot be delegated to the tool that lacks it

The ledger's summary: every containment charges the same underlying currency — a human holding position and certainty that the machine cannot hold. This is why the low-token/next-token problem is, ultimately, an argument about where humans remain load-bearing, not merely a fact about samplers.

Appendix L — Boundary Conditions and Honest Limits

Where the paper's claims stop, stated so they can be tested.

Claim	Holds when	Does not claim
Prior-token dominance	Local conditioning, general case	That long-range context never redirects — it can, where in-context evidence is dense enough
Laundering is total	Current transformer architectures	That no future architecture could carry confidence — it could; that's the point of §11's boundary
Drainage is directed toward corpus mass	Off-manifold generation	That every drift is harmful — drift toward the median is <i>helpful</i> when the median is what you wanted
Determinism ≠ cure	Proven via exact arithmetic	That determinism is useless — it is a debugging and reproducibility asset

Claim	Holds when	Does not claim
Sparse tail is inexhaustible	As long as original work mints new specificity	That scaling is pointless — it genuinely helps on-manifold work
Permanent for the architecture	Transformer forward pass, no rejection/confidence mechanism	That it’s permanent for <i>all</i> architectures — VDR-LLM-Prolog is the counter-construction

Appendix M — Registry Cross-Reference

Concept in this paper	Nearest treatment elsewhere	Relationship
Ballistic generation (the macro view of drainage)	[HOWL-LLM-3-2026]	This paper is the token-level cause
Cannot be fixed with more LLM	[HOWL-LLM-4-2026]	This paper names the specific unfixable (no confidence channel)
What the model cannot do / knows it can’t	[HOWL-LLM-5-2026]	The confidence-channel absence is <i>why</i> it cannot know
Session coherence engineering	[HOWL-LLM-6-2026]	Manufactured density as a session-structuring practice
Decoherency tumbler (per-edit erosion)	[HOWL-LLM-7-2026]	Per-token cause of per-edit erosion
Rejection step by construction	[HOWL-LLM-1-2026], VDR-LLM-Prolog series	The architectural fix outside this paper’s scope
Exact arithmetic / determinism	[HOWL-VDR-1-2026]+	The experiments proving determinism $\neq$ cure
Regression to the vulnerable mean	[HOWL-COMP-14-2026]	Drainage applied to security fixes
Manufactured density, one-pass, ownership	[HOWL-ENG-3-2026] App. F	Full practice specification of §11
Stillness vs. tumbling	[HOWL-COMP-14-2026]	Ownership-on-arrival is entry into stillness

[[HOWL-LLM-8-2026](#)] The Low-Token / Next-Token Problem. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM-8-2026>

[[HOWL-LLM-3-2026](#)] Riding the Rocket. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-3-2026>

[[HOWL-LLM-7-2026](#)] The Agentic Decoherency Tumbler Problem. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-7-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-COMP-14-2026](#)] Enterprise Toy Composition. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP-14-2026>

[[HOWL-ENG-3-2026](#)] Dynamic Engineering. Github: <https://github.com/ghowland/papers/tree/main/papers/ENG/HOWL-ENG-3-2026>

[[HOWL-LLM-1-2026](#)] Integer LLM with Prolog Knowledge Base. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-1-2026>

[[HOWL-LLM-2-2026](#)] Calibrate, Extract, Refine. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-2-2026>

[[HOWL-LLM-4-2026](#)] Incompatibility by Construction. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-4-2026>

[[HOWL-LLM-5-2026](#)] What I Cannot Do. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-5-2026>

[[HOWL-LLM-6-2026](#)] Session Coherence Structuring for Exploration and High Quality Extraction. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-6-2026>